

- 1. Problem Statement:** Build a Simple application flow to upload the image files and save the content in DB. Also preview them on the screen once successfully uploaded to DB.
- 2. Technology Stack:**
AWS stack or AZURE stack with any programming language of your choice, incorporating Gen AI – code companion tools
- 3. Features/Requirements:**

Feature/requirement	Feature /Requirement Description with Business rules and design constraints
Create the Database Models	Create a simple Image Database and save the file details.
Build API to upload and preview the images	1. Create APIs. • POST /upload to upload image details to DB • GET /preview/{imageId} to get image details and preview the content. 2. Secure API endpoints
Build UI screens to upload image and preview the image upon confirmation	This includes 1. file selection from the local directory 2. Validate the image type allowed liked jpg, png and gif. Others are not allowed. 3. Validate the image size to less than 2 MB. 4. Upload the image and call the API /upload 5. Showing the loader until confirmation is returned. 6. Upon confirmation pass the image id and get the image detail by calling API /preview/{imageId} 7. Preview the returned image in separate section in the same widow/tab along with the other image details given below.
Implement security measures	1. Use secure authentication mechanism like OAuth or JWT. 2. Ensure only authorized users can upload and preview the image 3. Securely manage API Keys 4. Implement Role Based Access Control
Code Quality and Code Coverage	1. Ensure code quality with static analysis tools 2. Code coverage for API should be 80-90%
Non-Functional requirements	1. Implement errors/exceptions handling mechanisms 2. Implement logging and monitoring to detect and troubleshoot issues 3. Implement rate limiting to prevent abuse of API endpoints 4. Response time < 30 seconds 5. Use containerization to package and deploy the application

	6. Improve maintainability with modular codebase, automated testing, CI/CD pipelines
--	--

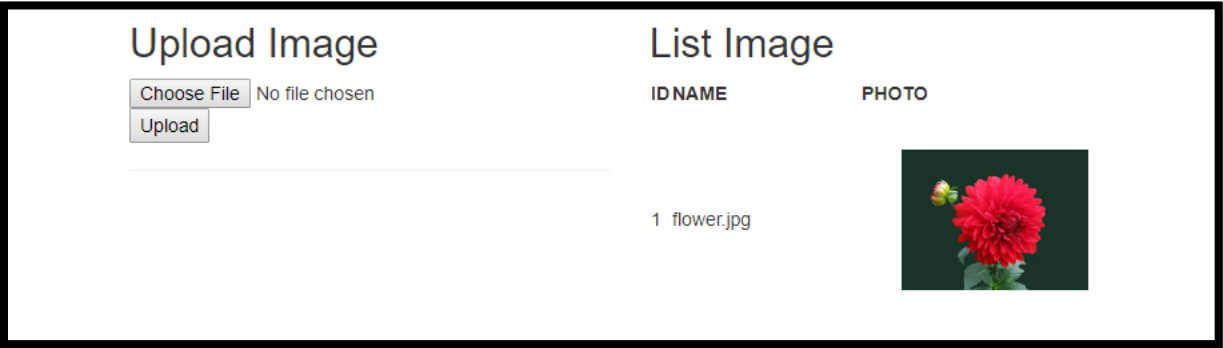
4. Key Points to be followed:

5. Table Structure:

Table Name: image_details

- ✓ Image_id - Primary key, unique identifier, auto increment
- ✓ Image_type – varchar [5]
- ✓ Image_path -varchar [200]
- ✓ Image_size -varchar [5]
- ✓ created_at: The date and time when the user was created.

6. Sample UI



7. Output to be generated with/without GitHub Copilot:

Phase 1:

1. Application Architecture & Deployment Architecture
2. High level design diagrams
3. Low level design diagrams: class diagrams, sequence diagrams, ER diagrams, component diagram, deployment view all mentioned **Low level design document** template
4. API Design document: API views, API design, keys and everything as per **API design document** template
5. Presentation

Phase 2: Execution (Code Companion Tools help)

1. Code Output
2. UI /Backend Demo
3. Code Quality and Code Coverage Report
4. All prompts (Failed as well Passed prompts)
5. Screenshots

Enhancement with this Use Case “Document Upload Portal” :

- **Auto-compression & optimization:** Reduce file size without losing quality to improve performance.
- **Duplicate detection:** Identify if the same image has already been uploaded.
- **Inline preview:** Show a quick preview before committing the upload.
- **Watermarking:** Automatically add watermarks to uploaded images for ownership protection.